

DOCUMENTAÇÃO DE LABORATÓRIO

Amazon EC2

Criar Instâncias pelo Console e pela AWS CLI — Host Bastion e Servidor Web

Serviço Principal Amazon EC2	Duração Aproximadamente 45 minutos	Métodos Usados Console + AWS CLI
--	--	--

1. Visão Geral

Neste laboratório, trabalhamos com dois métodos distintos para criar instâncias do Amazon EC2: o Console de Gerenciamento da AWS (interface gráfica) e a AWS CLI (linha de comando). Cada abordagem tem seu próprio conjunto de vantagens e casos de uso ideais.

A arquitetura final consiste em duas instâncias: um host bastion (criado pelo Console) e um servidor web (criado pela AWS CLI a partir do host bastion). O servidor web é configurado automaticamente durante a inicialização por um script de User Data que instala o Apache e implanta uma aplicação web.

1.1 Objetivos

- Iniciar uma instância EC2 como host bastion usando o Console de Gerenciamento da AWS.
- Conectar-se ao host bastion usando o EC2 Instance Connect (sem SSH).
- Usar a AWS CLI a partir do host bastion para criar uma segunda instância (servidor web).
- Verificar o funcionamento do servidor web pelo navegador.

1.2 Arquitetura do Laboratório

 Host Bastion Criado por: Console de Gerenciamento da AWS Função: Ponto de acesso seguro para executar comandos AWS CLI contra outros recursos da conta.	 Servidor Web Criado por: AWS CLI (a partir do host bastion) Função: Serve uma aplicação web instalada pelo script de User Data durante a inicialização.
--	--

1.3 Quando usar cada método?

Método	Ideal para	Limitações
Console Web	Instâncias rápidas ou temporárias, exploração do serviço	Propenso a erros humanos; difícil de repetir
AWS CLI / Script	Automação repetível e confiável, integração CI/CD	Requer conhecimento de parâmetros

AWS CloudFormation	Recursos relacionados, infraestrutura como código completa	Maior curva de aprendizado inicial
---------------------------	--	------------------------------------

2. Tarefa 1 — Criar o Host Bastion pelo Console

A primeira instância foi criada manualmente pelo Console de Gerenciamento da AWS, percorrendo as oito etapas do assistente de criação de instâncias.

2.1 Configurações Definidas

Parâmetro	Valor Configurado
Nome da instância	Bastion host
AMI	Amazon Linux 2 (HVM) – Quick Start
Tipo de instância	t3.micro
Par de chaves	Prosseguir sem par de chaves
VPC	Lab VPC
Sub-rede	Sub-rede pública (seleção padrão)
IP público automático	Habilitado (padrão)
Nome do grupo de segurança	Bastion security group
Descrição do grupo de segurança	Permit SSH connections
Armazenamento	8 GiB gp2 (padrão)
Perfil de instância IAM	Bastion-Role

2.2 Conceitos Importantes Aplicados

Conceito	Aplicação no Laboratório
AMI	Template do disco de inicialização. Usamos Amazon Linux 2, mantida e atualizada pela AWS.
Tipo de instância	t3.micro: instância de propósito geral com CPU intermitente, ideal para desenvolvimento e testes.
Grupo de segurança	Firewall virtual que controla o tráfego de entrada e saída. Criamos “Bastion security group”.
Perfil IAM	Bastion-Role: concede à instância permissão para chamar APIs da AWS via CLI, sem precisar configurar credenciais manualmente.
User Data	Script executado automaticamente na primeira inicialização. Usado na Tarefa 3 para instalar o Apache no servidor web.

💡 Por que usar o Bastion-Role em vez de credenciais?

Ao associar um perfil IAM (Instance Profile) à instância, ela recebe permissões temporárias automaticamente via metadados da instância. É muito mais seguro do que inserir Access Keys manualmente na instância — não há credenciais estáticas para vazarem ou gerenciar.

3. Tarefa 2 — Conectar ao Host Bastion via EC2 Instance Connect

Após a criação do host bastion, usamos o EC2 Instance Connect para abrir um terminal diretamente no navegador. Não é necessário par de chaves SSH nem cliente externo.

1. No Console EC2, selecionamos a instância **Bastion host** e clicamos em **Conectar-se**.
2. Na guia **EC2 Instance Connect**, clicamos em **Conectar-se**. O terminal foi aberto no navegador.

🔒 EC2 Instance Connect — Acesso Seguro sem Chaves

O EC2 Instance Connect injeta uma chave SSH temporária na instância via API da AWS, que expira em 60 segundos. Não é necessário gerenciar arquivos .pem ou .ppk. O acesso é controlado por políticas IAM e registrado no CloudTrail.

4. Tarefa 3 — Criar o Servidor Web pela AWS CLI

Com o terminal do host bastion aberto, executamos uma sequência de comandos para coletar os parâmetros necessários e criar a instância do servidor web automaticamente.

4.1 Etapa 1 — Recuperar o ID da AMI mais recente

Em vez de buscar o ID da AMI manualmente no Console, usamos o Parameter Store do AWS Systems Manager, que mantém sempre o ID da AMI mais atual do Amazon Linux 2.

```
# Determinar a zona de disponibilidade e exportar a região
AZ=`curl -s http://169.254.169.254/latest/meta-data/placement/availability-zone`
export AWS_DEFAULT_REGION=${AZ::-1}

# Buscar o ID da AMI mais recente do Amazon Linux 2
AMI=$(aws ssm get-parameters \
  --names /aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2 \
  --query 'Parameters[0].[Value]' --output text)
echo $AMI
```

💡 Por que usar o Parameter Store para a AMI?

As AMIs são atualizadas regularmente pela AWS com correções de segurança. Usar o Parameter Store garante que seu script sempre lance a versão mais recente automaticamente, sem precisar atualizar o ID da AMI no código.

4.2 Etapa 2 — Recuperar o ID da Sub-rede Pública

Buscamos o ID da sub-rede pelo nome da tag, sem precisar copá-lo manualmente do console:

```
SUBNET=$(aws ec2 describe-subnets \
  --filters 'Name=tag:Name,Values=Public Subnet' \
  --query Subnets[].SubnetId --output text)
echo $SUBNET
```

4.3 Etapa 3 — Recuperar o ID do Grupo de Segurança

Da mesma forma, buscamos o grupo de segurança pelo nome — esse grupo já existia no ambiente e permite tráfego HTTP de entrada:

```
SG=$(aws ec2 describe-security-groups \
  --filters Name=group-name,Values=WebSecurityGroup \
  --query SecurityGroups[].GroupId --output text)
echo $SG
```

4.4 Etapa 4 — Baixar o Script de User Data

O script de User Data é executado automaticamente na primeira inicialização da instância. Ele instala o Apache, baixa e implanta a aplicação web:

```
wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/[...]/UserData.txt
cat UserData.txt # visualizar o conteúdo
```

O script realiza internamente:

- Instalação do servidor web Apache (httpd).
- Download do arquivo .zip com a aplicação web.
- Descompactação e instalação da aplicação.

4.5 Etapa 5 — Lançar a Instância do Servidor Web

Com todos os parâmetros coletados, executamos o comando `run-instances`:

```
INSTANCE=$( \
  aws ec2 run-instances \
  --image-id $AMI \
  --subnet-id $SUBNET \
  --security-group-ids $SG \
  --user-data file:///home/ec2-user/UserData.txt \
  --instance-type t3.micro \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=Web Server}]' \
  --query 'Instances[*].InstanceId' \
  --output text \
)
echo $INSTANCE
```

Parâmetros utilizados:

Flag	O que faz
<code>--image-id \$AMI</code>	ID da AMI obtido do Parameter Store
<code>--subnet-id \$SUBNET</code>	Sub-rede pública onde a instância será criada
<code>--security-group-ids \$SG</code>	Grupo de segurança que permite HTTP
<code>--user-data file://...</code>	Script executado na primeira inicialização
<code>--instance-type t3.micro</code>	Tipo de instância a ser criada
<code>--tag-specifications</code>	Atribui a tag Name=Web Server
<code>--query</code>	Retorna apenas o Instance ID na saída
<code>--output text</code>	Formata a saída como texto simples

4.6 Etapa 6 — Verificar o Status da Instância

Monitoramos o estado da instância até ela estar pronta para receber requisições:

```
# Todas as informações da instância em JSON
aws ec2 describe-instances --instance-ids $INSTANCE

# Apenas o estado atual (pending / running)
aws ec2 describe-instances --instance-ids $INSTANCE \
  --query 'Reservations[].Instances[].State.Name' --output text
```

Executamos o segundo comando repetidamente até o retorno ser

```
running
```

4.7 Etapa 7 — Testar o Servidor Web

Recuperamos o DNS público da instância e abrimos no navegador para confirmar o funcionamento:

```
aws ec2 describe-instances --instance-ids $INSTANCE \
  --query Reservations[].Instances[].PublicDnsName --output text
```

DNS Público da Instância — Ocultado

O nome DNS público da instância (formato `ec2-XX-XX-XX-XX.us-west-2.compute.amazonaws.com`) é um dado de acesso sensível e foi omitido neste documento.

Servidor Web Verificado

Ao acessar o DNS público no navegador, a página da aplicação web foi exibida corretamente, confirmando que o Apache foi instalado e a aplicação implantada com sucesso pelo script de User Data.

5. Desafios Opcionais

O laboratório incluiu dois desafios opcionais de troubleshooting em uma instância chamada Misconfigured Web Server, que já existia no ambiente com erros propositais.

5.1 Desafio 1 — Corrigir o Acesso via EC2 Instance Connect

O objetivo era identificar por que a conexão via EC2 Instance Connect falhava nessa instância e corrigir o problema.

✗ Sintoma

EC2 Instance Connect falhava ao tentar conectar à instância Misconfigured Web Server.

☑ Diagnóstico e Correção

O grupo de segurança da instância não permitia tráfego SSH (porta 22). A regra de entrada foi adicionada para resolver o problema.

5.2 Desafio 2 — Corrigir o Servidor Web

Após corrigir o acesso, o próximo passo era descobrir por que o servidor web não estava respondendo no navegador.

✗ Sintoma

O DNS público da instância não retornava a página esperada no navegador.

☑ Diagnóstico e Correção

O serviço Apache (httpd) não estava em execução na instância. Iniciamos o serviço manualmente com `sudo systemctl start httpd`.

6. Lições Aprendidas



O Console é ótimo para explorar opções e criar recursos rapidamente. A CLI é essencial para automação, scripts e pipelines de CI/CD.



Usar um Instance Profile IAM na instância elimina a necessidade de configurar credenciais estáticas. É sempre a abordagem preferível para instâncias que precisam chamar APIs da AWS.



O script de User Data é a forma padrão de configurar uma instância durante sua inicialização. Executa apenas uma vez (na primeira inicialização) e é ideal para bootstrapping de servidores.



A abordagem de recuperar parâmetros dinâmicos (AMI ID, Subnet ID, SG ID) pela CLI, em vez de hardcodá-los, torna os scripts portáteis e sempre atualizados.



Os desafios opcionais reforçam dois problemas clássicos de troubleshooting: grupo de segurança com porta fechada e serviço não iniciado. Ambos são facilmente diagnosticados com acesso à instância e inspeção dos logs.



Variáveis de ambiente (AMI, SUBNET, SG, INSTANCE) são perdidas se a sessão do EC2 Instance Connect cair. Em scripts de produção, sempre persista esses valores em arquivo ou passe-os como parâmetros explícitos.

7. Resumo dos Comandos CLI Utilizados

Comando	Finalidade
<code>aws ssm get-parameters</code>	Buscar o ID da AMI mais recente no Parameter Store
<code>aws ec2 describe-subnets</code>	Descobrir o ID da sub-rede pública pelo nome da tag
<code>aws ec2 describe-security-groups</code>	Descobrir o ID do WebSecurityGroup pelo nome
<code>aws ec2 run-instances</code>	Criar e iniciar a instância do servidor web
<code>aws ec2 describe-instances</code>	Verificar o estado e detalhes da instância criada
<code>aws ec2 describe-instances --query ...State.Name</code>	Monitorar apenas o status da instância
<code>aws ec2 describe-instances --query ...PublicDnsName</code>	Obter o DNS público para testar o servidor web

8. Recursos Adicionais

- Executar sua instância — docs.aws.amazon.com/AWSEC2/latest/UserGuide/LaunchingAndUsingInstances.html
- EC2 Instance Connect — docs.aws.amazon.com/AWSEC2/latest/UserGuide/Connect-using-EC2-Instance-Connect.html
- Amazon EC2: Dados de usuário e scripts de shell — docs.aws.amazon.com/AWSEC2/latest/UserGuide/user-data.html
- AWS CLI — docs.aws.amazon.com/cli/latest/reference/ec2/
- AWS Training and Certification — aws.amazon.com/training

Documentação elaborada após a conclusão do laboratório. Dados sensíveis (IDs de instância, DNS público, AMI IDs, Subnet IDs, SG IDs) foram propositalmente omitidos.